

Product Requirements Document (PRD)

Tax-Aware Portfolio Rebalancing Engine

1. Core Architecture & Security (Zero-PII)

To fulfill the strict "Zero Personal Identifiable Information" requirement, the application must be designed to process data without retaining it.

- **Local-First Processing:** The app should ideally be built as a Single Page Application (SPA) using frameworks like React, Vue, or Svelte, processing all CSV/PDF parsing on the client side via the browser.
- **Ephemeral State:** If backend processing is required for the AI agent or heavy projections, data should be passed via secure API calls with no database persistence.
- **No API Broker Linking:** Do not use services like Plaid to link accounts, as this inherently introduces PII and security risks. All data ingestion must rely on user-provided, anonymized files or manual entry.

2. Data Input & Ingestion Requirements

The system needs to capture the user's current portfolio state seamlessly.

- **File Parsing:** The app must ingest CSV files (standard broker exports) and PDF statements. (Note: PDF parsing is notoriously brittle due to varying broker formats; standardizing a CSV template is highly recommended for v1).
- **User Inputs Required:**
 - **Accounts:** Type (Brokerage-Taxable, Tax-Deferred [IRA, 401k], Tax-Free [Roth, HSA]).
 - **Holdings:** Ticker symbol, Quantity, Purchase Price (Cost Basis), Current Value, Capital Gains/Losses to date, and Yield.
 - **Target Allocation:** Desired percentages for major asset classes (e.g., Total US Stock, Total International, Total Bond, REITs).
 - **Projection Horizon:** User-specified timeframe in years and months for risk/reward modeling.

3. Core Calculation Engine

- **Real-Time Data Integration:** The app must fetch real-time or end-of-day market values for inputted tickers using an external finance API (e.g., Alpha Vantage, Yahoo Finance API).
- **Asset Allocation Analysis:** The system must calculate and display two primary views:
 - *Blended View:* Total allocation percentages rolled up across *all* accounts.

- *Account-Level View*: Allocation isolated within each specific account.
- **Risk/Reward Projections**: Use a deterministic compound interest model or a Monte Carlo simulation based on historical asset class returns to project the portfolio forward. Formula basis: $FV = PV \times (1 + r)^t$ (where PV is the current balance, r is expected return based on risk profile, and t is the user-specified time horizon).

4. Tax-Efficient Rebalancing Engine

This is the core logic of the application, aligning closely with the principles of simple, low-cost index investing and asset location framework. The goal of asset location is to improve after-tax returns by deciding where to hold different types of investments across account types.

The Rebalancing Algorithm should follow these rules:

- **Step 1: Calculate the Delta**: Compare current blended allocations against the user's target % allocations to determine buy/sell amounts for each asset class.
- **Step 2: Prioritize Trades in Tax-Advantaged Accounts**: When rebalancing, prioritize trades inside IRAs and Roth accounts to avoid realizing taxable gains.
- **Step 3: Asset Location Mapping (The Placement Rules)**:
 - *Tax-Deferred Accounts (Traditional IRA, 401k)*: Direct tax-inefficient holdings (taxable bonds, REITs, high-turnover funds) into tax-deferred accounts when possible.
 - *Tax-Free Accounts (Roth, HSA)*: Direct higher-expected-return equity tilts and long-horizon growth holdings into Roth-style accounts, subject to your comfort with volatility in those accounts.
 - *Taxable Brokerage Accounts*: Taxable accounts are used for broad equity index funds, tax-efficient ETFs, municipal bonds, and other holdings where you benefit from long-term capital gains treatment and control over when to realize gains.
- **Step 4: Taxable Account Adjustments**: Use cash flows and, when appropriate, tax-loss harvesting in taxable accounts to move gradually toward your targets.

5. AI Financial Advisor & Snapshot Feature

- **Snapshots**: The user must be able to hit a "Save Snapshot" button. Since there is no database, this should generate a downloadable, encrypted JSON file or an exported PDF report capturing the exact "moment in time" portfolio state, allocations, and proposed trades.

- **AI Insight Agent:** Integrate with an LLM via API (e.g., OpenAI or Anthropic).
 - *Prompt Engineering:* Pass the anonymized portfolio snapshot (current allocation vs. target, tax-efficiency grade) to the LLM. Instruct the LLM to adopt the persona of a pragmatic, low-cost financial advisor to generate 3-4 actionable insights (e.g., "You are holding a REIT in your taxable account, which creates tax drag. Consider moving this exposure to your IRA during your next rebalance.").

6. Tips for the Programmer

- **Start with a Standardized CSV Schema:** Build a downloadable .csv template that users can fill out. This bypasses the immediate nightmare of parsing 50 different PDF statement formats from various brokerages.
- **Tagging System:** Implement an internal database/mapping system that automatically tags inputted tickers with their asset class (e.g., VTI = Broad US Equity, BND = Taxable Bond, VNQ = REIT) so the rebalancing engine knows *how* to treat them based on the tax-efficiency rules above.
- **UI/UX:** Use visual band-rebalancing charts. Show the user their current allocation bar chart next to their target allocation bar chart, with clear "Buy X" and "Sell Y" instructions mapped to specific accounts.